portfolio-web — explications

Un site portfolio statique en HTML, CSS et jQuery qui présente des projets décrits dans un fichier JSON, avec recherche, filtres, fiche détaillée et thème sombre.

Jean-Emmanuel Jacquier

Sommaire

1. Le projet en bref
   - Ce que fait le projet
   - Les compétences qu'il montre
   - Un exemple concret
2. Lancer et tester
3. Architecture
4. Le code pas à pas
   - 4.1 Avant l'affichage : le thème mémorisé
   - 4.2 Le module de filtrage, utilisable partout
   - 4.3 Le démarrage de app.js et le chargement des données
   - 4.4 Le filtrage : une fonction pure
   - 4.5 Le rendu des cartes
   - 4.6 Les filtres par catégorie et par compétence
   - 4.7 Le nuage de compétences et les compteurs
   - 4.8 La fiche détaillée (modale)
   - 4.9 Le bouton de thème
5. Ce qu'on apprend en le codant soi-même
   - 5.1 La délégation d'événements
   - 5.2 Séparer la logique du DOM pour pouvoir la tester
   - 5.3 Ne pas tout redessiner à chaque touche : le debounce
   - 5.4 `$.getJSON` et le protocole `file://`
   - 5.5 Un thème sombre sans flash
   - 5.6 Une modale accessible
   - 5.7 `.text()` plutôt que `.html()` : éviter l'injection de code
   - 5.8 Tester comme un utilisateur avec Playwright
   - 5.9 Une CI qui teste avant de publier
   - 5.10 Comparer des textes en français : accents et Unicode
   - 5.11 Moins d'animations quand on le demande
6. Pour aller plus loin
7. Glossaire

1. Le projet en bref

Ce que fait le projet

`portfolio-web` est le site personnel de Jean-Emmanuel Jacquier, étudiant en informatique à Epitech Lyon. C'est une seule page web qui présente ses projets GitHub sous forme de cartes. Le visiteur peut rechercher un projet par mot-clé (sans se soucier des accents ni des majuscules), filtrer par catégorie, cliquer sur une compétence comme « Docker » pour ne voir que les projets qui l'utilisent, ouvrir la fiche détaillée d'un projet à la souris ou au clavier, et basculer entre thème clair et thème sombre.

Le site est **statique** : il n'y a ni serveur applicatif ni base de données. Le navigateur télécharge `index.html`, une feuille de style, deux scripts et le fichier de données `data/projets.json`, puis construit la page en JavaScript.

Il sert de vitrine pour une recherche d'alternance : un recruteur doit pouvoir trouver en quelques secondes « les projets en Python » ou « ce qui touche à l'infrastructure ». Comme les projets sont décrits dans un fichier JSON, en ajouter un ne demande aucune modification du code.

Les compétences qu'il montre

- **HTML** sémantique et accessible : modale `role="dialog"` avec piège à focus, `inert`, `aria-*` (`index.html`) ;
- **CSS** moderne : variables, thème clair/sombre, grille responsive (`css/style.css`) ;
- **jQuery** : AJAX, délégation d'événements, animations coupées par `prefers-reduced-motion` (`js/app.js`) ;
- **JavaScript** : fonctions pures, normalisation Unicode (`js/filtres.js`) ;
- **tests** : 9 tests `node --test` et 34 tests Playwright (`tests/`) ;
- **CI/CD** : GitHub Actions et déploiement GitHub Pages (`.github/workflows/ci.yml`).

Un exemple concret

La logique de filtrage ne dépend pas du navigateur : on peut l'appeler depuis Node.js pour voir ce que la page affichera. Ici, une recherche sans accent retrouve un projet dont le titre en contient un, puis on affiche les deux compétences les plus fréquentes :

```
node -e '
const F = require("./js/filtres.js");
const p = require("./data/projets.json");
console.log(F.filtrerProjets(p, { recherche: "emissions" }).map(x => x.titre));
console.log(F.compterCompetences(p).slice(0, 2));'
```

```
# sortie réelle
[ 'Émissions de CO₂ en R' ]
[ { nom: 'Python', nombre: 5 }, { nom: 'Docker', nombre: 3 } ]
```

Dans la page, taper « emissions » dans la zone de recherche laisse exactement cette carte visible, et « Python » est le mot le plus gros du nuage de compétences.

2. Lancer et tester

Le détail se trouve dans le `README.md`. Le site doit être servi par HTTP (la section 5.4 explique pourquoi un double-clic sur `index.html` ne suffit pas) :

```
npm start                        # python3 -m http.server, puis ouvrir http://
127.0.0.1:8080
npm install
npm test                         # 9 tests unitaires (node --test), ~0,1 s
npx playwright install chromium
npm run test:e2e                 # 34 tests de bout en bout (Playwright)
```

`npm start` ne fait que lancer le serveur intégré à Python 3 : rien n'est téléchargé. Les 17 scénarios de `tests/portfolio.spec.js` sont joués deux fois, sur un écran d'ordinateur et sur un téléphone simulé (Pixel 7), d'où 34 tests. Si un Chromium est déjà installé, la variable `CHROMIUM_PATH` permet de l'utiliser (voir `playwright.config.js`). À chaque `push` sur `main`, `.github/workflows/ci.yml` relance tous les tests puis, s'ils passent, publie le site sur GitHub Pages.

3. Architecture

```
push sur main --> GitHub Actions (ci.yml)
                    tests : npm test, npm run test:e2e
                    déploiement : index.html css js data --> GitHub Pages

Navigateur
  index.html     script inline du <head> : applique le thème mémorisé
  css/style.css  variables de couleur, thème sombre
  jQuery (CDN)   vérifié par l'attribut integrity
  js/filtres.js  crée window.Filtres (fonctions pures, sans DOM)
  js/app.js      $(function () { ... }) quand le DOM est prêt
      |
      |  $.getJSON("data/projets.json")
      v
  projets[] --> afficherCategories()  afficherProjets()  afficherNuage()
                                                |
                 Filtres.filtrerProjets(projets, criteres)
                                                ^
       criteres = { recherche, categorie, competence }
                                                ^
  Événements : saisie #recherche, clic .filtre, clic .etiquette / .mot
               clic ou Entrée sur .carte --> ouvrirModale()
               Échap, fond, bouton Fermer --> fermerModale()
```

Le principe central : **un seul état** (`criteres`) et **une seule fonction de rendu** (`afficherProjets`). Chaque action de l'utilisateur modifie `criteres`, puis la grille est redessinée à partir de zéro.

Fichier	Rôle
`index.html`	Squelette : sections à remplir, modale cachée, script inline du thème
`css/style.css`	Variables de couleur, thème sombre, grille de cartes, modale, mobile
`js/filtres.js`	Fonctions pures : `normaliser`, `filtrerProjets`, `compterCompetences`, `categories`
`js/app.js`	Code jQuery : chargement, rendu, événements, modale, thème, compteurs
`data/projets.json`	Les 12 projets (titre, résumé, détails, compétences, catégorie, dépôt)
`tests/filtres.test.js`	Tests `node --test` des filtres et de la validité du JSON
`tests/portfolio.spec.js`	Tests Playwright : recherche, filtres, modale, thème, mobile
`playwright.config.js`	Serveur de test (`npm start`), profils « ordinateur » et « mobile »
`.github/workflows/ci.yml`	Tests puis déploiement GitHub Pages
`docs/`, `explications-pdf.yml`	Ce document, son script PDF et le workflow qui le régénère

4. Le code pas à pas

On suit le parcours complet, de l'ouverture de la page au clic sur une compétence.

4.1 Avant l'affichage : le thème mémorisé

Tout en haut de `index.html`, un petit script s'exécute **avant** que le navigateur ne dessine la page :

```
<script>
  // Appliqué avant l'affichage pour éviter un flash de thème clair.
  try {
    var t = localStorage.getItem("theme");
    if (t) document.documentElement.dataset.theme = t;
  } catch (e) {}
</script>
```

`localStorage.getItem("theme")` lit le choix fait lors d'une visite précédente (`"clair"` ou `"sombre"`), ou `null`. La ligne suivante pose l'attribut `data-theme="sombre"` sur `<html>`, auquel réagit le CSS (voir 5.5). Le `try/catch` protège contre les navigateurs où le stockage est interdit : la page fonctionne quand même, sans mémoire du thème.

En bas de la page, trois scripts sont ensuite chargés dans l'ordre : jQuery depuis le CDN `code.jquery.com` (avec un attribut `integrity` qui contient l'empreinte SHA-256 attendue, pour que le navigateur refuse un fichier modifié), puis `js/filtres.js`, puis `js/app.js`, qui a besoin des deux premiers.

4.2 Le module de filtrage, utilisable partout

`js/filtres.js` fonctionne à la fois dans le navigateur et dans Node.js :

```
(function (exports) {
  "use strict";
  function normaliser(texte) { /* ... */ }
  function filtrerProjets(projets, criteres) { /* ... */ }
  // ...
  exports.normaliser = normaliser;
  exports.filtrerProjets = filtrerProjets;
  exports.compterCompetences = compterCompetences;
  exports.categories = categories;
})(typeof module !== "undefined" ? module.exports : (window.Filtres = {}));
```

Tout le fichier est une fonction appelée immédiatement, qui range ses fonctions dans l'objet `exports` qu'on lui passe. La dernière ligne choisit cet objet : dans Node.js, la variable `module` existe et les fonctions vont dans `module.exports`, ce qui permet `require("../js/filtres.js")` dans les tests ; dans le navigateur, on crée `window.Filtres = {}`. Le test `typeof module !== "undefined"` est la seule façon sûre de tester une variable qui peut ne pas exister : `module !== undefined` lèverait une `ReferenceError` dans le navigateur.

4.3 Le démarrage de app.js et le chargement des données

`js/app.js` est entièrement enveloppé dans `$(function () { ... })`, le raccourci jQuery pour « exécuter quand le DOM est prêt ». Il déclare l'état (`projets`, `criteres`, `dernierFocus`), puis les gestionnaires d'événements, et seulement à la fin charge les données :

```
$.getJSON("data/projets.json")
  .done(function (donnees) {
    projets = donnees;
    afficherCategories();
    afficherProjets();
    afficherNuage();
  })
  .fail(function () {
    $grille.html("<p class='vide'>Impossible de charger les projets.</p>");
  });
```

`$.getJSON` envoie une requête HTTP en arrière-plan (AJAX) et transforme la réponse en tableau JavaScript. La requête est **asynchrone** : les fonctions passées à `.done` (succès) ou `.fail` (échec) sont appelées plus tard, à l'arrivée de la réponse. Les gestionnaires sont donc posés avant que la moindre carte n'existe ; cela fonctionne grâce à la délégation d'événements (5.1). Le `$` devant `$grille` est une convention qui signale une variable contenant un objet jQuery.

4.4 Le filtrage : une fonction pure

Le cœur du site est `filtrerProjets` dans `js/filtres.js` :

```
function filtrerProjets(projets, criteres) {
  var recherche = normaliser(criteres.recherche || "");
  return projets.filter(function (p) {
    if (criteres.categorie && criteres.categorie !== "tous"
        && p.categorie !== criteres.categorie) {
      return false;
    }
    if (criteres.competence && p.competences.indexOf(criteres.competence) === -1) {
      return false;
    }
    if (!recherche) {
      return true;
    }
    var texte = normaliser([p.titre, p.resume, p.details]
      .concat(p.competences).join(" "));
    return recherche.split(/\s+/).every(function (mot) {
      return texte.indexOf(mot) !== -1;
    });
  });
}
```

(Deux lignes sont coupées pour la mise en page.)

- `criteres.recherche || ""` remplace une recherche absente par une chaîne vide, ce qui autorise l'appel `filtrerProjets(projets, {})` dans les tests.
- Les trois critères se cumulent : catégorie, **puis** compétence, **puis** recherche.
- Pour la recherche, titre, résumé, détails et compétences sont réunis en un seul texte normalisé (sans accents, en minuscules, voir 5.10).
- `recherche.split(/\s+/)` découpe la saisie en mots ; `every` exige que **chaque** mot soit présent. « python discord coran » ne garde ainsi que AlHudaBot, ce que vérifie `tests/filtres.test.js`.
- La fonction ne modifie rien et ne touche pas au DOM : c'est une fonction **pure**.

4.5 Le rendu des cartes

`afficherProjets` dans `js/app.js` transforme le résultat du filtrage en HTML :

```
function afficherProjets() {
  var visibles = Filtres.filtrerProjets(projets, criteres);
  $grille.empty();
  visibles.forEach(function (p, i) {
    var $carte = $("<article class='carte' tabindex='0'>")
      .attr("data-id", p.id)
      .attr("aria-label", "Voir le détail de " + p.titre)
      .append($("<span class='categorie'>").text(LIBELLES[p.categorie] || p.categorie))
      .append($("<h3>").text(p.titre))
      .append($("<p>").text(p.resume))
      .append($("<ul class='etiquettes'>").append(etiquettes(p.competences)));
    $carte.appendTo($grille);
    // Fondu en cascade, sauf animations coupées : delay() ne tient pas compte de
$.fx.off.
    if (!$.fx.off) $carte.hide().delay(i * 40).fadeIn(200);
  });
  $(".vide").prop("hidden", visibles.length > 0).text(projets.length
    ? "Aucun projet ne correspond à cette recherche."
    : "Aucun projet à afficher pour le moment.");
  $(".filtre-actif").prop("hidden", !criteres.competence)
    .find("strong").text(criteres.competence || "");
}
```

- `$grille.empty()` : on redessine tout plutôt que de calculer quelles cartes ajouter ou retirer. Avec douze projets, c'est instantané et beaucoup plus simple.
- `tabindex='0'` rend l'`<article>` atteignable avec Tab (voir 5.6).
- Tous les textes passent par `.text(...)`, jamais par `.html(...)` (voir 5.7).
- `LIBELLES[p.categorie] || p.categorie` affiche « Bots Discord » pour `bots`, ou la valeur brute si aucun libellé n'est prévu.
- `.hide().delay(i * 40).fadeIn(200)` cache chaque carte insérée, puis la fait apparaître en fondu avec 40 ms de décalage par rang : effet de cascade. Quand les animations sont coupées (`$.fx.off`, voir 5.11), la carte reste simplement visible.
- Les deux dernières instructions affichent un message seulement si la liste est vide (« Aucun projet ne correspond… », ou « Aucun projet à afficher… » si `data/projets.json`

lui-même est vide), et le bandeau « Filtré par … » seulement si une compétence est choisie.

4.6 Les filtres par catégorie et par compétence

Les boutons de catégorie sont créés par `afficherCategories` à partir de `Filtres.categories(projets)` (catégories dans l'ordre d'apparition, sans doublon). Un clic met à jour `criteres.categorie` (lu avec `.attr("data-categorie")`, voir 5.1), passe `aria-pressed` à `true` sur le bouton choisi (ce qui sert aux lecteurs d'écran et au CSS `.filtre[aria-pressed="true"]`), puis appelle `afficherProjets()`.

Les étiquettes des cartes, celles de la modale et les mots du nuage portent tous un attribut `data-competence`. Un seul gestionnaire les couvre :

```
function filtrerParCompetence(nom) {
  criteres.competence = nom;
  fermerModale();
  afficherProjets();
  document.getElementById("projets").scrollIntoView();
  $(".filtre-actif")[0].focus({ preventScroll: true });
}

$(document).on("click", ".etiquette, .mot", function (e) {
  e.stopPropagation();
  filtrerParCompetence($(this).attr("data-competence"));
});
```

`e.stopPropagation()` empêche le clic de remonter jusqu'à la carte qui contient l'étiquette ; sans lui, cliquer sur « Docker » dans une carte ouvrirait aussi la fiche du projet. `fermerModale()` ne fait rien si la modale est déjà fermée. `scrollIntoView()` ramène la section « Projets » en haut de l'écran : le défilement est doux grâce à `scroll-behavior: smooth` sur `<html>`, et `scroll-padding-top: 70px` le fait s'arrêter sous l'en-tête collant (voir 5.11).

Le focus suit le défilement : il va sur le bandeau « Filtré par … », qui porte `tabindex="-1"` dans `index.html` (focalisable par le code, mais pas avec Tab). Un lecteur d'écran peut alors lire le filtre appliqué, et le Tab suivant atteint « tout afficher », puis les cartes. `preventScroll: true` évite que `focus()` ne lance son propre défilement vers le bandeau. Ce n'est pas un détail : voir 5.6. Symétriquement, « tout afficher » cache le bandeau qui le contient, donc son gestionnaire renvoie le focus au champ de recherche.

4.7 Le nuage de compétences et les compteurs

`afficherNuage` crée un bouton `.mot` par compétence. `Filtres.compterCompetences` les renvoie de la plus fréquente à la moins fréquente (départage alphabétique avec `localeCompare(..., "fr")`), donc `stats[0].nombre` est le maximum, et la taille de police varie de 0,9 rem à 1,8 rem avec `0.9 + 0.9 * s.nombre / max`.

Si `data/projets.json` est un tableau vide, `stats[0]` n'existe pas, et lire `stats[0].nombre` lèverait une `TypeError` qui arrêterait tout le rendu (compteurs compris). La fonction teste donc `stats.length` et remplace la phrase d'aide par « Aucune compétence à afficher pour le moment. » ; un test Playwright sert un JSON vide pour le vérifier.

Les compteurs « 12 projets » et « N technologies » utilisent une astuce jQuery dans `animerCompteur` : `$({ valeur: 0 }).animate({ valeur: cible }, { duration: 900, ... })`. Ici `$()` enveloppe un simple objet JavaScript, pas un élément HTML ; `.animate` fait varier sa propriété `valeur` de 0 à `cible`, la fonction `step` écrit à chaque étape `Math.round(this.valeur)` dans la page, et `complete` garantit un nombre final exact.

4.8 La fiche détaillée (modale)

```
function ouvrirModale(id) {
  var p = projets.find(function (x) { return x.id === id; });
  if (!p) return;
  dernierFocus = document.activeElement;
  $modale.find("#modale-titre").text(p.titre);
  $modale.find(".modale-details").text(p.details);
  $modale.find(".modale-competences").empty().append(etiquettes(p.competences));
  $modale.find(".modale-lien").attr("href", p.depot);
  $modale.stop(true, true).prop("hidden", false).hide().fadeIn(150);
  $arrierePlan.prop("inert", true);
  $modale.find(".modale-fermer").trigger("focus");
  $("body").addClass("sans-defilement");
}
```

`dernierFocus` mémorise l'élément qui avait le focus (la carte) pour le lui rendre à la fermeture. La modale existe déjà dans `index.html` : on remplit seulement ses zones. `.prop("hidden", false).hide().fadeIn(150)` retire l'attribut `hidden`, cache aussitôt avec jQuery, puis fait apparaître en fondu ; le `.stop(true, true)` qui précède est expliqué en 5.6. `$arrierePlan` regroupe l'en-tête, `main` et le pied de page : l'attribut `inert` les rend inertes (ni clic, ni focus, ni lecture par un lecteur d'écran) tant que la modale est ouverte. Le focus va sur le bouton « Fermer », et la classe `sans-defilement` (`overflow: hidden`) bloque le défilement de l'arrière-plan. La fermeture et l'usage au clavier sont détaillés en 5.6.

4.9 Le bouton de thème

La fonction `themeSombre()` de `js/app.js` détermine le thème affiché : il est sombre si `data-theme` vaut `"sombre"`, **ou** si aucun choix n'est enregistré et que `window.matchMedia("(prefers-color-scheme: dark)").matches` est vrai. Au clic sur `.bouton-theme`, on pose l'autre valeur sur `<html>` et on l'enregistre avec `localStorage.setItem("theme", nouveau)` (dans un `try/catch`), pour que le script inline de 4.1 la réapplique à la prochaine visite.

Le bouton est un interrupteur : son nom accessible est fixe (`aria-label="Thème sombre"`) et son état est porté par `aria-pressed`, que `majBoutonTheme()` recalcule au chargement, après

chaque clic et quand la préférence du système change (événement `change` de `matchMedia`). Un lecteur d'écran annonce ainsi le nom, le rôle de bouton bascule et l'état (activé ou non). Le nom ne doit pas changer avec l'état : « Passer au thème clair », annoncé « activé », serait contradictoire. L'icône, purement visuelle, vient du CSS : `content: var(--icone-theme)` sur `.bouton-theme::before`, la variable valant « ☾ » dans le thème clair et « ☀ » dans le sombre, dans les mêmes blocs que les couleurs (5.5).

5. Ce qu'on apprend en le codant soi-même

Cette partie rassemble les notions pratiques qu'on découvre en écrivant ce projet, en général après s'être heurté au problème. Pour chacune : le contexte, l'erreur naïve, la solution retenue dans le code.

5.1 La délégation d'événements

Contexte. Les cartes, les étiquettes et les boutons de catégorie n'existent pas dans `index.html` : ils sont créés par JavaScript après le chargement du JSON, et recréés à chaque filtrage par `$grille.empty()`.

L'approche naïve. On écrit naturellement `$(".carte").on("click", ...)`. Au moment où cette ligne s'exécute, `$.getJSON` n'a pas encore répondu : `$(".carte")` ne trouve **aucun** élément, et le gestionnaire n'est attaché à rien. Aucune erreur n'apparaît ; le clic ne fait simplement rien. Et même placée dans `.done`, cette ligne ne suffirait pas : le premier filtrage détruirait les cartes et leurs gestionnaires. Un essai dans Chromium avec jQuery 3.7.1 le confirme, seul le gestionnaire délégué réagit :

```
$(".x").on("click", () => n++);              // attaché à rien
$("#g").on("click", ".x", () => n += 10);    // délégué au conteneur
$("#g").append("<button class=x>");
$(".x").trigger("click");                    // n vaut 10, pas 11
```

La bonne façon, dans le projet. On attache le gestionnaire à un parent **stable** et on précise un sélecteur en deuxième argument :

```
$grille.on("click", ".carte", function () { ouvrirModale($(this).attr("data-id")); });
$(".categories").on("click", ".filtre", function () { /* ... */ });
$(document).on("click", ".etiquette, .mot", function (e) { /* ... */ });
```

Le clic sur une carte « remonte » (on dit qu'il se propage, bubbling) jusqu'à `#grille`, qui existe depuis le début. jQuery vérifie alors si l'élément cliqué, ou l'un de ses ancêtres, correspond à `.carte`, et appelle la fonction avec `this` égal à cette carte. Les étiquettes sont déléguées à `document` car elles apparaissent à trois endroits (cartes, modale, nuage).

Corollaire : comme l'événement remonte, un clic sur une étiquette **dans** une carte atteindrait aussi le gestionnaire de la carte. D'où le `e.stopPropagation()` de 4.6.

Le clavier a produit un vrai bug du même genre. Le gestionnaire `keydown` de `.carte` ouvre la fiche sur Entrée ou Espace et appelle `e.preventDefault()`. Or l'appui sur Entrée dans une étiquette remonte, lui aussi, jusqu'à la carte : la fiche s'ouvrait, et le `preventDefault()` annulait le clic que le navigateur aurait déclenché sur le bouton, si bien que le filtre n'était jamais appliqué. `stopPropagation()` dans le gestionnaire de clic n'y pouvait rien, puisque ce clic n'avait jamais lieu. La correction consiste à ne traiter que les touches adressées à la carte elle-même :

```
$grille.on("keydown", ".carte", function (e) {
  if (e.target !== this) return;   // touche frappée dans une étiquette
  // ...
});
```

Piège voisin : `.data()` convertit les attributs `data-*`, et `data-id="42"` devient le **nombre** 42 ; la comparaison `x.id === id` d'`ouvrirModale` échouait alors. Le projet lit donc `data-id`, `data-categorie` et `data-competence` avec `.attr()`, qui renvoie toujours une chaîne. Un test Playwright sert un projet d'identifiant `"42"` pour le vérifier.

> **À retenir :** pour des éléments créés dynamiquement, on attache l'écouteur à un conteneur qui existe déjà, avec `.on("click", "selecteur", ...)`. Un sélecteur qui ne trouve rien ne provoque aucune erreur, il ne fait simplement rien. Un gestionnaire délégué reçoit aussi les événements des descendants : comparer `e.target` et `this` avant d'appeler `preventDefault()`.

5.2 Séparer la logique du DOM pour pouvoir la tester

Contexte. La règle « quels projets afficher ? » est la partie la plus susceptible de contenir des bugs : combinaison de trois critères, accents, plusieurs mots.

L'approche naïve. On écrit le filtre dans le gestionnaire jQuery, en parcourant les cartes avec `$(".carte").each(...)` et en cachant celles dont `.text()` ne contient pas la recherche. Cela fonctionne, mais le seul moyen de le tester est d'ouvrir un navigateur : chargé dans Node.js, ce code s'arrête sur `ReferenceError: $ is not defined`. De plus, la recherche porterait sur le texte **affiché**, pas sur le champ `details`, qui n'apparaît que dans la modale.

La bonne façon, dans le projet. Toute la logique est dans `js/filtres.js`, qui ne connaît ni jQuery ni `document` : des tableaux en entrée, un tableau en sortie. Le motif de 4.2 (`typeof module !== "undefined" ? module.exports : window.Filtres`) permet de charger le **même fichier** dans la page et dans les tests :

```
const { filtrerProjets } = require("../js/filtres.js");
const projets = require("../data/projets.json");

test("filtre par compétence", () => {
  const go = filtrerProjets(projets, { competence: "Go" });
  assert.deepEqual(go.map((p) => p.id), ["go-uptime"]);
});
```

Les 9 tests de `tests/filtres.test.js` s'exécutent en une fraction de seconde avec `node --test`, sans dépendance à installer. Ils vérifient aussi les données : chaque projet a tous ses champs, un lien vers `github.com/jejcqr/`, et aucun identifiant n'est en double. Une faute de frappe dans `data/projets.json` est donc détectée par la CI avant la mise en ligne.

> **À retenir :** isoler les règles métier dans des fonctions pures (données en entrée, résultat en sortie, sans DOM) les rend testables en millisecondes. Le code jQuery ne fait plus que relier ces fonctions à la page.

5.3 Ne pas tout redessiner à chaque touche : le debounce

Contexte. La recherche filtre en direct pendant la frappe. Chaque redessin vide la grille et relance les fondus des cartes.

L'approche naïve.

```
$("#recherche").on("input", function () {
  criteres.recherche = $(this).val();
  afficherProjets();
});
```

Taper « docker » déclenche six redessins en moins d'une seconde. Les animations `delay(i * 40).fadeIn(200)` se chevauchent : les cartes clignotent, et sur un téléphone peu puissant la saisie devient saccadée.

La bonne façon, dans le projet.

```
var minuterie;
$("#recherche").on("input", function () {
  var valeur = $(this).val();
  clearTimeout(minuterie);
  minuterie = setTimeout(function () {   // évite de tout redessiner à chaque touche
    criteres.recherche = valeur;
    afficherProjets();
  }, 150);
});
```

Chaque frappe annule le minuteur précédent (`clearTimeout`) et en programme un nouveau. Le redessin n'a donc lieu que lorsque l'utilisateur a fait une pause de 150 ms. Ce motif s'appelle debounce (anti-rebond). La valeur est lue **immédiatement** (`var valeur = $(this).val()`), car dans la fonction différée `this` ne désigne plus le champ de saisie.

L'événement `input` est préférable à `keyup` : il se déclenche aussi pour un collage à la souris, une dictée vocale ou un clic sur la croix d'effacement d'un champ `type="search"`.

> **À retenir :** pour réagir à une saisie continue, on attend une courte pause avec `clearTimeout` + `setTimeout` plutôt que de travailler à chaque frappe.

5.4 `$.getJSON` et le protocole `file://`

Contexte. Le site charge ses données avec `$.getJSON("data/projets.json")`.

L'approche naïve. On double-clique sur `index.html`. Le navigateur l'ouvre avec une adresse `file:///.../index.html`, et la page affiche seulement « Impossible de charger les projets. ». La console de Chromium explique :

```
Access to XMLHttpRequest at 'file:///home/claude/portfolio-web/data/projets.json'
from origin 'null' has been blocked by CORS policy: Cross origin requests are only
supported for protocol schemes: chrome, chrome-extension, chrome-untrusted, data,
http, https, isolated-app.
```

Pour des raisons de sécurité, une page ouverte depuis le disque n'a pas d'« origine » (elle vaut `null`) et n'a pas le droit de lire d'autres fichiers locaux par requête AJAX. Sinon, un fichier HTML téléchargé pourrait lire tout le disque de l'utilisateur et l'envoyer sur Internet.

La bonne façon, dans le projet. On sert le dossier par HTTP, même en local :

```
npm start      # python3 -m http.server 8080 --bind 127.0.0.1
```

`playwright.config.js` lance la même commande (option `webServer`) avant les tests, et c'est la situation réelle sur GitHub Pages. Le code prévoit en outre le cas d'échec avec `.fail(...)` : l'utilisateur voit un message plutôt qu'une page vide.

Deux détails utiles :

- le chemin `data/projets.json` est **relatif** (sans `/` initial). Il fonctionne donc aussi bien à la racine d'un domaine que dans un sous-dossier comme `https://jejcqr.github.io/portfolio-web/` ;
- `.done` et `.fail` ne sont appelés qu'**après** la réponse. Tout code qui a besoin des projets doit être appelé depuis `.done`, pas écrit juste après `$.getJSON`.

> **À retenir :** une page qui fait des requêtes AJAX doit être servie par HTTP, même pendant le développement. Une requête peut échouer : il faut toujours prévoir le cas d'erreur.

5.5 Un thème sombre sans flash

Contexte. Le site respecte la préférence du système (mode sombre de Windows, macOS, Android…), mais l'utilisateur peut aussi imposer son choix avec le bouton.

L'approche naïve. On applique le thème enregistré dans `app.js` :

```
$(function () {
  $("html").attr("data-theme", localStorage.getItem("theme"));
});
```

Résultat : à chaque chargement, la page s'affiche d'abord en clair pendant une fraction de seconde, puis bascule en sombre. Ce clignotement (flash of unstyled content) vient de ce que `app.js` est chargé en fin de page, après jQuery depuis le CDN, alors que le navigateur a déjà commencé à dessiner.

La bonne façon, dans le projet. Le script inline du `<head>` (4.1) pose `data-theme` **avant** le premier affichage ; il est minuscule et n'a besoin d'aucune bibliothèque. Ensuite, les couleurs sont des **variables CSS** redéfinies selon le thème dans `css/style.css` (extrait simplifié) :

```
:root { --fond: #f7f7f5; --texte: #1c1f26; color-scheme: light; }
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="clair"]) { --fond: #111318; --texte: #e8eaee; }
}
:root[data-theme="sombre"] { --fond: #111318; --texte: #e8eaee; }
```

Le reste du CSS n'utilise que ces variables (`background: var(--fond)`) : un seul attribut sur `<html>` change toutes les couleurs d'un coup. La règle `:root:not([data-theme="clair"])` exprime : « si le système est en sombre, appliquer le sombre, **sauf** si l'utilisateur a explicitement choisi le clair ». La propriété `color-scheme` indique au navigateur d'adapter aussi ses propres éléments (barres de défilement, champ de recherche). Enfin, `localStorage` est toujours entouré d'un `try/catch`, car son accès peut lever une exception lorsque le stockage est bloqué.

> **À retenir :** tout ce qui change l'apparence dès le premier affichage doit être appliqué dans le `<head>`, avant le chargement des scripts lourds. Des variables CSS permettent de changer de thème en modifiant un seul attribut.

5.6 Une modale accessible

Contexte. La fiche détaillée s'affiche par-dessus la page. Elle doit être utilisable au clavier et compréhensible par un lecteur d'écran.

L'approche naïve. Une `<div>` montrée avec `.show()` et des cartes en `<div>` cliquables. Avec Tab, on n'atteint jamais une carte (une `<div>` n'est pas focalisable), donc la fiche est

inaccessible sans souris ; un lecteur d'écran ne sait pas qu'une fenêtre s'est ouverte ; après fermeture, le focus retombe au début de la page.

Un autre piège, purement CSS : la modale est cachée avec l'attribut HTML `hidden`, mais le CSS lui donne `display: grid` pour la centrer. Or une règle CSS d'auteur l'emporte sur le style par défaut de `hidden` : la modale resterait visible en permanence. Le projet ajoute donc explicitement :

```
.modale { position: fixed; inset: 0; display: grid; place-items: center; }
.modale[hidden] { display: none; }
```

La bonne façon, dans le projet.

```
<div class="modale" id="modale" role="dialog" aria-modal="true"
     aria-labelledby="modale-titre" hidden>
```

- `role="dialog"` et `aria-modal="true"` annoncent une boîte de dialogue ; `aria-labelledby` lui donne pour nom le titre du projet.
- Les cartes ont `tabindex='0'` et un `aria-label`, et réagissent à Entrée et Espace :

```
$grille.on("keydown", ".carte", function (e) {
  if (e.target !== this) return;
  if (e.key === "Enter" || e.key === " ") {
    e.preventDefault();
    ouvrirModale($(this).attr("data-id"));
  }
});
```

Le `preventDefault()` empêche Espace de faire défiler la page ; la première ligne est expliquée en 5.1.

- À l'ouverture, le focus va sur le bouton « Fermer ». La touche Échap, écoutée sur `document`, appelle `fermerModale`, qui rend le focus à `dernierFocus` (la carte d'origine) et ne remet l'attribut `hidden` qu'à la **fin** du fondu, dans la fonction de rappel de `fadeOut(150, ...)`.
- `aria-modal="true"` promet que rien d'autre n'est utilisable : il faut le tenir. Tant que la modale est ouverte, l'en-tête, `main` et le pied de page reçoivent l'attribut `inert` (4.8), et un gestionnaire fait boucler Tab du dernier élément au premier, et Maj+Tab du premier au dernier :

```
$modale.on("keydown", function (e) {
  if (e.key !== "Tab") return;
  var $elements = $modale.find("a[href], button:not(:disabled)");
  var premier = $elements.get(0), dernier = $elements.get(-1);
  if (e.shiftKey && document.activeElement === premier) {
    e.preventDefault();
    dernier.focus();
  } else if (!e.shiftKey && document.activeElement === dernier) {
    e.preventDefault();
    premier.focus();
  }
});
```

`inert` seul ne suffit pas : après le dernier élément, Tab ferait sortir le focus de la page, vers l'interface du navigateur. À la fermeture, `inert` est retiré **avant** de rendre le focus à la carte, sinon `focus()` sur un élément inerte serait sans effet.

Deux pièges réels sont apparus en testant au clavier. D'abord, un élément focalisé qui disparaît du DOM emporte le focus avec lui : après un filtre lancé depuis une étiquette de la modale, `fermerModale` rendait le focus à la carte d'origine, aussitôt détruite par `afficherProjets()`, et le focus retombait sur `<body>`, au tout début de la page. Le filtre place désormais le focus lui-même (4.6). Ensuite, appuyer sur Échap puis tout de suite sur Entrée rouvrait la fiche **pendant** le fondu de fermeture : le rappel de `fadeOut` remettait ensuite `hidden`, et la fiche restait affichée mais marquée cachée, impossible à refermer. `$modale.stop(true, true)` termine le fondu en cours (rappel compris) avant de rouvrir.

> **À retenir :** un élément interactif doit être atteignable au clavier, avoir un nom lisible et rendre le focus là où il l'a pris ; une modale doit garder le focus et rendre l'arrière-plan inerte. Avant de détruire l'élément qui a le focus, décider où le mettre. Un `display` défini en CSS annule l'attribut `hidden` : il faut ajouter `[hidden] { display: none; }`.

5.7 `.text()` plutôt que `.html()` : éviter l'injection de code

Contexte. Tous les textes affichés viennent de `data/projets.json`. Aujourd'hui, ce fichier est écrit par l'auteur du site ; demain, il pourrait venir d'une API ou d'un formulaire.

L'approche naïve. Construire les cartes par concaténation de chaînes :

```
$grille.append("<article class='carte'><h3>" + p.titre + "</h3>"
  + "<p>" + p.resume + "</p></article>");
```

Si un titre contient `<img src=x onerror=alert(document.cookie)>`, le navigateur crée une vraie image dont l'erreur de chargement exécute le code. C'est une faille **XSS** (cross-site scripting). Moins grave mais plus courant : un résumé comme « Utilise la balise

» perdrait le mot `<main>`, que le navigateur prendrait pour une vraie balise.

La bonne façon, dans le projet. Les éléments sont créés vides, puis remplis avec `.text()` ou `.attr()` :

```
.append($("<h3>").text(p.titre))
.append($("<p>").text(p.resume))
```

`.text()` insère la valeur comme du texte brut : les caractères `<` et `>` sont affichés tels quels, jamais interprétés. Le seul `.html()` du projet est dans le cas d'erreur de `$.getJSON`, avec une chaîne fixe écrite dans le code.

> **À retenir :** une donnée ne doit jamais être insérée comme du HTML. On crée la structure avec des balises fixes, puis on y place les données avec `.text()` ou `.attr()`.

5.8 Tester comme un utilisateur avec Playwright

Contexte. Les tests unitaires vérifient le filtrage, mais pas que le clic sur « Docker » dans le nuage produise bien le bon nombre de cartes, ni que la page ne déborde pas sur un téléphone.

L'approche naïve. Tester à la main, ou écrire des tests qui patientent un temps fixe (`await page.waitForTimeout(1000)`) avant de compter les cartes avec `.count()`. Un tel test est lent et instable : si la machine de CI est chargée, les fondus durent plus longtemps et le comptage tombe au mauvais moment.

La bonne façon, dans le projet. `tests/portfolio.spec.js` utilise des assertions qui **réessaient** automatiquement jusqu'à ce qu'elles soient vraies (ou qu'un délai maximal soit atteint) :

```
test("les catégories filtrent les cartes", async ({ page }) => {
  await page.click(".filtre[data-categorie='bots']");
  await expect(page.locator(".carte")).toHaveCount(2);
});
```

Trois autres choix rendent la suite fiable :

- **Pas de dépendance au réseau.** Si le CDN de jQuery était lent ou injoignable, tous les tests échoueraient. Le `beforeEach` intercepte donc la requête avec `page.route("https://code.jquery.com/**", ...)` et répond avec `route.fulfill` en servant `node_modules/jquery/dist/jquery.min.js`. Cela ne marche que parce que la version npm est exactement la 3.7.1 : sinon l'empreinte `integrity` (4.1) ne correspondrait plus et le navigateur refuserait le script.
- **Les attentes viennent des données.** Le test « affiche tous les projets » compare à `projets.length` lu dans le JSON : ajouter un projet ne casse pas le test. À l'inverse, quand un cas limite l'exige, `page.route("**/data/projets.json", ...)` sert des données fabriquées (une liste vide, un identifiant `"42"`).

- **Les préférences de l'utilisateur sont simulées.** `page.emulateMedia({ colorScheme })` et `test.use({ reducedMotion: "reduce" })` jouent les tests du thème et de `prefers-reduced-motion` (5.11) sans toucher au système.
- **Deux appareils.** `playwright.config.js` déclare les profils `ordinateur` (Desktop Chrome) et `mobile` (Pixel 7). Le test « pas de défilement horizontal » vérifie que `scrollWidth` ne dépasse pas la largeur de l'écran, un défaut fréquent sur mobile.

> **À retenir :** un test de bout en bout doit attendre un **état** (« il y a deux cartes »), jamais une **durée**. Il doit aussi isoler la page des services externes pour donner le même résultat à chaque exécution.

5.9 Une CI qui teste avant de publier

Contexte. Le site est mis en ligne automatiquement à chaque `push` sur `main`.

L'approche naïve. Publier tout le dépôt dès qu'on pousse. Une virgule oubliée dans `data/projets.json` met alors en ligne un site vide, sans que personne ne soit prévenu, et les tests et la configuration sont publiés avec le site.

La bonne façon, dans le projet. Dans `.github/workflows/ci.yml` :

- le job `tests` exécute `npm ci` (installation exacte des versions de `package-lock.json`), `npm test`, puis les 34 tests Playwright ;
- le job `deploiement` déclare `needs: tests` : il ne démarre que si les tests ont réussi, et seulement sur `main` (`if: github.ref == 'refs/heads/main' && ...`) ;
- seul le dossier `_site`, qui contient `index.html`, `css`, `js` et `data`, est publié.

Les tests unitaires lisent le vrai `data/projets.json` : un JSON mal formé fait échouer `require("../data/projets.json")` dès le chargement du fichier de test, et le déploiement est bloqué.

> **À retenir :** une mise en ligne automatique doit dépendre du succès des tests et ne publier que les fichiers nécessaires. `npm ci` garantit que la CI utilise exactement les mêmes versions que le développeur.

5.10 Comparer des textes en français : accents et Unicode

Contexte. Un recruteur qui tape « emissions » ou « interpreteur » sur un clavier QWERTY doit trouver « Émissions de CO_2 en R » et le mini-shell (« Interpréteur de commandes »).

L'approche naïve. Passer les deux textes en minuscules et comparer :

```
"Émissions".toLowerCase().includes("emissions")   // false
```

« é » et « e » restent deux caractères différents, et une table de remplacement (`é -> e`, `ç -> c`…) serait forcément incomplète.

La bonne façon, dans le projet. Unicode prévoit qu'un caractère accentué peut s'écrire de deux façons : en un seul caractère (« é », U+00E9) ou en deux, la lettre de base suivie d'un accent dit « combinant » (« e » U+0065 + « ◌́ » U+0301). La méthode `normalize("NFD")` force la seconde forme ; il suffit ensuite de supprimer tous les accents combinants, qui occupent la plage U+0300 à U+036F :

```
function normaliser(texte) {
  return String(texte)
    .normalize("NFD")
    .replace(/[\u0300-\u036f]/g, "")
    .toLowerCase()
    .trim();
}
```

Dans `js/filtres.js`, la classe de caractères contient directement les deux caractères combinants au lieu des séquences `\u0300` et `\u036f` ; l'effet est identique, mais la forme échappée est plus lisible et ne risque pas d'être abîmée par un éditeur. Dans Node.js, `"é".length` vaut 1 mais `"é".normalize("NFD").length` vaut 2 : « e » puis l'accent.

Trois détails complètent la fonction :

- `String(texte)` évite une erreur si on lui passe un nombre ou `undefined` ;
- `trim()` retire les espaces au début et à la fin, ce que vérifie le test `normaliser(" Émissions CO₂ ")` qui doit donner `"emissions co₂"` ;
- la normalisation est appliquée **des deux côtés** : à la saisie et au texte du projet. Le chiffre « ₂ » n'est pas un accent et reste intact.

Pour le tri, en revanche, le projet n'enlève pas les accents : `compterCompetences` utilise `a.nom.localeCompare(b.nom, "fr")`, qui classe les mots selon les règles du français (« é » rangé avec « e », et non après « z » comme le ferait une comparaison de codes de caractères).

> **À retenir :** pour comparer des textes sans tenir compte des accents, on décompose avec `normalize("NFD")` puis on retire la plage U+0300 à U+036F. Pour trier, on utilise `localeCompare` avec la langue voulue.

5.11 Moins d'animations quand on le demande

Contexte. Certaines personnes (troubles vestibulaires, migraines, simple confort) demandent à leur système de réduire les animations. Le navigateur l'expose avec la requête média `prefers-reduced-motion: reduce`.

L'approche naïve. Une règle CSS qui coupe les transitions :

```css
@media (prefers-reduced-motion: reduce) {
  * { transition: none !important; scroll-behavior: auto !important; }
}
```

Elle est utile (survol des cartes), mais les fondus en cascade et les compteurs sont des `.animate()` jQuery, qui modifient les styles à chaque image depuis JavaScript. Le CSS n'en sait rien et ils continuaient de s'exécuter. Le défilement doux était lui aussi un `.animate({ scrollTop })` jQuery, et le gestionnaire des liens du menu appelait `e.preventDefault()` pour le lancer. Effet de bord : le navigateur ne traitait plus l'ancre, donc ne déplaçait pas le point de départ de la navigation au clavier. Après « Projets », la page avait défilé mais le Tab suivant repartait du menu.

La bonne façon, dans le projet. jQuery a un interrupteur global, `$.fx.off` : à `true`, chaque animation saute directement à son état final et appelle quand même sa fonction `complete`. `js/app.js` le règle d'après la même requête média, et le met à jour si la préférence change pendant la visite :

```js
var moinsDeMouvement = window.matchMedia("(prefers-reduced-motion: reduce)");
$.fx.off = moinsDeMouvement.matches;
moinsDeMouvement.addEventListener("change", function (e) { $.fx.off = e.matches; });
```

Le défilement doux est, lui, confié au CSS : `html { scroll-behavior: smooth; }`, que la règle ci-dessus ramène à `auto` sous `prefers-reduced-motion`. Les liens du menu n'ont plus aucun JavaScript : le navigateur suit l'ancre, met l'adresse à jour (`#projets`), s'arrête sous l'en-tête grâce à `scroll-padding-top` et place le point de départ de Tab dans la section visée. Un test Playwright le vérifie : après Entrée sur « Projets », Tab atteint le champ de recherche. Le filtre par compétence utilise `scrollIntoView()` (4.6), qui suit la même règle CSS.

Le compteur affiche alors directement « 12 » et le défilement vers les projets est immédiat. Seul `.delay()` y échappe : c'est un simple minuteur, qui aurait encore fait apparaître les cartes une à une ; `afficherProjets` ne l'utilise donc que si les animations sont actives (4.5). Le test Playwright correspondant observe toutes les valeurs écrites dans le compteur (une seule attendue) et vérifie, dans le même tour de boucle que le clic, que toutes les cartes sont visibles et que la page a défilé.

> **À retenir :** `prefers-reduced-motion` doit couper **toutes** les animations, y compris celles pilotées en JavaScript ; avec jQuery, `$.fx.off = true` suffit pour `.animate()` et les fondus, mais pas pour `.delay()`. Un défilement doux en CSS respecte la préférence et laisse le navigateur gérer les ancres et le focus.

6. Pour aller plus loin

Du plus simple au plus ambitieux :

1. **Annoncer le nombre de résultats.** Pendant la frappe, un lecteur d'écran ne sait pas combien de cartes restent. Piste : un paragraphe `aria-live="polite"` que `afficherProjets()` remplit avec « 3 projets », et un test Playwright qui lit son texte.
2. **Vérifier l'accessibilité automatiquement.** Les tests actuels couvrent les défauts déjà rencontrés, pas les autres (contraste insuffisant, nom manquant…). Piste : ajouter `@axe-core/playwright` et un test qui analyse la page, en thème clair et en thème sombre, modale ouverte et fermée, et exige zéro violation.
3. **Mémoriser les filtres dans l'adresse.** Piste : écrire `criteres` dans `location.hash` à chaque changement et le relire avant le premier `afficherProjets()`, pour pouvoir partager un lien déjà filtré.
4. **Remplacer la modale par l'élément natif `<dialog>`.** Sa méthode `showModal()` rend déjà l'arrière-plan inerte et gère Échap. Piste : supprimer le code correspondant de `js/app.js` et vérifier que les tests de la modale (focus piégé, réouverture rapide) restent verts.
5. **Compléter `data/projets.json` depuis l'API GitHub.** Piste : un script Node.js lancé en CI interroge `https://api.github.com/users/jejcqr/repos` et ajoute la date de mise à jour et les étoiles, ce qui permet un tri « plus récents d'abord ».
6. **Réécrire sans jQuery.** Piste : `fetch` au lieu de `$.getJSON`, `addEventListener` et `event.target.closest(...)` pour la délégation, transitions CSS pour les fondus. Les tests Playwright, qui ne regardent que le comportement visible, doivent rester verts sans modification.

7. Glossaire

Terme	Définition
AJAX	Requête HTTP faite en arrière-plan par une page, sans la recharger
Asynchrone	Opération dont le résultat arrive plus tard ; le code continue sans l'attendre
`aria-*`	Attributs HTML qui décrivent rôle et état d'un élément aux technologies d'assistance
Bubbling	Propagation d'un événement de l'élément cliqué vers tous ses ancêtres
CDN	Réseau de serveurs qui distribue des fichiers publics, ici jQuery
CI/CD	Tests et mise en ligne automatiques à chaque modification du dépôt
CORS	Règles du navigateur qui limitent les requêtes d'une origine vers une autre
Debounce	Attendre une pause dans une série d'événements avant d'agir
Délégation	Écouteur posé sur un parent stable, valable pour des descendants créés plus tard
DOM	Représentation en mémoire de la page, que JavaScript lit et modifie
Focus	Élément qui reçoit actuellement les frappes du clavier
Fonction pure	Fonction dont le résultat ne dépend que de ses arguments, sans effet de bord
`localStorage`	Stockage clé/valeur du navigateur, conservé entre les visites
Modale	Fenêtre affichée par-dessus la page, qui bloque l'arrière-plan
NFD	Normalisation Unicode qui sépare une lettre accentuée en lettre de base plus accent
Origine	Protocole + domaine + port d'une page ; une page `file://` a l'origine `null`
Playwright	Outil qui pilote un vrai navigateur pour des tests de bout en bout
SRI	Attribut `integrity` : le navigateur vérifie l'empreinte d'un fichier externe
Site statique	Fichiers servis tels quels, sans programme exécuté côté serveur
Variable CSS	Propriété personnalisée (`--fond`) réutilisée avec `var(--fond)`
XSS	Faille où un contenu injecté dans une page est exécuté comme du code